



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026-27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 12

Team No.: 1

Project Title: Linux-Based Alarm Clock Using System Programming Concepts

Team Members

Roll Number	Student Name	Signature
2520090206	K. Laxmi Lahari	K. L. Lahari
2520090174	S. Rishil	S. Rishil
2520090082	S. Anjani Shreyaj	S. Anjani

1. Abstract

The proposed project is a Linux-based Alarm Clock implemented using C programming and Operating System concepts. The system allows users to set an alarm for a specific time and notifies the user when the specified time is reached. Unlike a conventional application-based alarm clock, this project focuses on demonstrating Linux/POSIX system programming mechanisms. The project uses process management, signals, timers, and system calls to implement the alarm functionality. The alarm() system call is used to schedule a signal after a specified period, while SIGALRM is handled to notify the user when the alarm time is reached.

The proposed system provides a simple terminal-based interface through which users can set, view, and cancel alarms. The system continuously waits for the specified time without unnecessary processing and generates an alarm notification when the timer expires. The expected outcome is a functional Linux-based alarm clock that demonstrates practical applications of processes, signals, timers, and system calls, helping to understand how the Operating System manages processes and asynchronous events using Linux/POSIX mechanisms.

2. Problem Statement

Many alarm applications are implemented as general-purpose GUI or mobile applications, which may hide the underlying Operating System mechanisms involved in timing and event notification. The objective of this project is to develop a Linux-based terminal Alarm Clock that demonstrates how Operating System and system programming concepts can be used to schedule and trigger an alarm. The system accepts an alarm time from the user, calculates the required waiting period, and notifies the user when the specified time is reached. The project focuses on Linux/POSIX mechanisms such as signals, timers, process management, and system calls rather than developing a generic GUI or database application, directly aligning with the requirement that the core implementation demonstrate Operating Systems and Linux/POSIX concepts.

3. Objectives

1. To design and implement a Linux-based terminal Alarm Clock using C programming.
2. To use Linux/POSIX system calls and signals for scheduling and triggering alarms.
3. To demonstrate Operating System concepts such as process management, signal handling, and timer-based events.
4. To provide a simple and reliable system for setting, viewing, and cancelling alarms through a terminal interface.

4. Proposed Methodology

The system is implemented in C on a Linux platform and is built around a terminal-based menu that lets the user set a new alarm, view active alarms, or cancel an existing one. When a user sets an alarm, the program computes the number of seconds remaining until the target time and schedules a SIGALRM signal using the alarm() (or setitimer()) system call. A custom signal handler, registered with sigaction(), is triggered when the signal is delivered, printing an alarm notification to the terminal.

While waiting, the process suspends itself using pause() or sleep() instead of continuously polling the system clock, keeping CPU usage negligible. Process management concepts are used to run the waiting alarm as a background process, so the terminal remains free for other commands; cancelling an alarm is handled by resetting the timer with alarm(0) or sending a termination signal to the alarm process. The design keeps the flow linear: input time -> validate and compute delay -> arm the timer -> block until SIGALRM -> notify -> return to the menu.

The source code will be compiled using the GCC compiler and executed from the Linux terminal. The implementation will be tested with different scenarios, including setting a single alarm, setting multiple alarms, and cancelling an alarm before it triggers, to verify that the timer, signal handling, and process management components work correctly together.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
alarm() system call	Schedules a SIGALRM signal to be delivered to the process after a specified number of seconds.
setitimer()	Provides a finer-grained interval timer as an alternative to alarm() for scheduling the countdown.
signal() / sigaction()	Registers a custom handler for SIGALRM so the program can respond when the alarm time is reached.
SIGALRM	Standard POSIX signal used to notify the process that the requested alarm time has elapsed.
pause() / sleep()	Suspends process execution while waiting for the alarm signal, avoiding busy-waiting and unnecessary CPU usage.
fork()	Creates a child process so the alarm wait can run in the background while the terminal remains free.
Process ID (PID)	Identifies and tracks the background alarm process so it can later be located and cancelled.

kill()	Sends a signal to the running alarm process to cancel it before the alarm time is reached.
Linux Shell	Used to launch the program and interact with it through terminal commands.
Standard Input/Output	Used to accept the alarm time from the user and display alarm notifications and status messages.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520090206	K. Laxmi Lahari	Design and implementation of the core alarm logic, including alarm()/setitimer() scheduling and the SIGALRM signal handler.
2520090174	S. Rishil	Development of the terminal-based menu interface for setting, viewing, and cancelling alarms, including input validation.
2520090082	S. Anjani Shreyaj	Process management implementation (running the alarm as a background process), testing, documentation, and GitHub repository setup.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system platform providing POSIX-compliant APIs for signals, timers, and process management.
C / C++	Programming language used to implement the alarm clock logic and interface with Linux system calls.
GCC	Compiler used to build and compile the C source code.
Bash Terminal	Environment used to run, test, and interact with the alarm clock program.
Make / Makefile	Used to automate the compilation and build process.
VS Code / gedit	Text editor used for writing and debugging the source code.
Git / GitHub	Used to maintain, manage, and submit the project source code and documentation.

8. Expected Outcome

The project will result in a working Linux terminal-based Alarm Clock that allows a user to set, view, and cancel one or more alarms through simple commands. On reaching the specified time, the program will reliably generate an alarm notification using SIGALRM without any busy-waiting or unnecessary CPU usage while idle.

Beyond the alarm functionality itself, the project is expected to give practical, hands-on understanding of how the Linux/POSIX kernel manages processes, delivers and handles signals, and schedules timer-based asynchronous events -- concepts that are central to Operating Systems and Systems Programming.

Different test cases will be performed using a single alarm, multiple concurrent alarms, and an alarm cancelled before it triggers, to verify the correctness of the implementation. The final submission will include the source code, build instructions, terminal output, and screenshots demonstrating each test case.

9. GitHub Repository

GitHub Repository Link: [https://github.com/rishill10/KLH-CSIT-2026-27-01-Alarm-](https://github.com/rishill10/KLH-CSIT-2026-27-01-Alarm-Clock)

Clock Repository Created:

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name: Raghupathi M

Signature: _____

Remarks: _____

-

Faculty Use

Project Approved: _____

☒ Yes ☐ No

Date: _____ 02/09/2026

Faculty Signature: _____